Zewail City
of Science, Technology and Innovation

Communication and Information Engineering

Embedded Systems
Course Code: CIE 349

VMO

Autonomous Home Pet and Safety Monitoring Robot

Technical Documentation

Project Team:

Matthew Nader	202300103
Omar Tamer Samir	202300713
Mohamed Gadwal	202301231
Amr Emad Mohamed	202301016
Mohamed Magdy	202101520

Contents

1 Introduction

VMO is an interactive, autonomous home pet robot engineered to engage dynamically with users using computer vision and a localized sensor array. Drawing stylistic inspiration from classic compact robotic companions, VMO acts simultaneously as an engaging household companion and a safety monitor. By continually logging environmental factors, the system alerts users to anomalies, bridging consumer robotics with proactive smart-home defense.

The platform is built around a heterogeneous, multi-tier computing hierarchy. Hard real-time sensor polling and motor actuation are governed by an 8-bit PIC microcontroller; asynchronous task distribution and wireless communication are handled by a dual-core ESP32-S3 running FreeRTOS; and high-level cognitive perception—including real-time computer vision, ROS 2 navigation, and cloud telemetry—runs inside a containerized Linux environment on a Raspberry Pi 4. A consumer Android smartphone mounted to the chassis serves as VMO's expressive face, voice interface, and local network hotspot.

2 Hardware Architecture

2.1 Distributed System Processing Tiers

The VMO system relies on a heterogeneous processing topology that decouples high-latency, computation-heavy algorithms from time-critical, hardware-level execution loops. This guarantees real-time deterministic reactivity under tight hardware constraints.

- **Muscle Layer (Hard Real-Time):** Governed by an 8-bit Microchip PIC16F877A microcontroller, dedicated strictly to physical sensor polling and deterministic motor drive execution.
- **Nervous System (Task Distribution):** Governed by a dual-core Espressif ESP32-S3 running FreeRTOS, providing hardware isolation, asynchronous queue buffering, and cross-platform communication serialization.
- **Brain Layer (High-Level Cognitive Perception):** Governed by a Raspberry Pi 4 (2 GB RAM) executing a headless Linux containerized environment. This tier handles spatial vector extraction, autonomous path evaluation, and distributed ROS 2 microservice pipelines.
- **Face UI Layer (Passive Expressions):** Governed by an Android smartphone mounted directly to the chassis, processing high-level emotional feedback states without consuming power from the core processing units.

2.2 Bill of Materials (BOM)

Processing and Communication

- **Main Controller:** Microchip PIC16F877A 8-bit microcontroller — primary 5 V hardware logic and real-time sensor polling.
- **IoT Coprocessor:** ESP32-S3-N8R2 3.3 V module — computer vision coordination and Wi-Fi cloud connectivity.
- **Signal Bridging:** Bi-directional Logic Level Converters providing a noise-isolated UART bridge between the 3.3 V and 5 V system domains.
- **System Clock:** 20 MHz Crystal Oscillator providing a stable reference frequency for the primary microcontroller.

Power Distribution and Management

- **Main Supply Rail:** 11.1 V 3S Lithium-Polymer (LiPo) battery pack with a dedicated BMS protection module.
- **Emergency Cutoff:** Physical high-current hardware kill switch acting as an absolute system interrupter.
- **Overcurrent Safety:** 10 A inline fast-blow fuse protecting the primary battery feed.
- **Logic Regulation:** Dual high-efficiency buck converter modules generating independent 5 V and 3.3 V power rails.
- **Power Telemetry:** INA226 digital power monitor providing real-time I^2C voltage and current measurements.
- **Actuation Driver:** L298N H-Bridge dual motor driver for differential drive locomotion.

Sensors and Peripherals

- **Environmental Safety:** MQ-2 hazardous gas and smoke sensor on a 10-bit ADC channel, with a critical leak threshold at ADC > 400.
- **Climate Telemetry:** DHT11 digital temperature and relative humidity sensor.
- **Spatial Obstacle Array:** 4-channel HC-SR04 ultrasonic sensors in a Front-Left-Right (FLR) configuration for path clearing, plus a dedicated rear sensor for reversing safety.
- **Odometry Tracking:** Dual high-speed optical wheel encoders mounted on the drive shafts for precise relative positioning.

Passives, Protection, and Indicators

- **Visual Diagnostics:** Multi-color LED array (White, Red, Yellow, Green, Blue) for immediate hardware status indication.
- **Reverse Voltage Isolation:** 15SQ045 high-current Schottky diode protecting downstream silicon.
- **Filtering and Decoupling:** 470 µF bulk capacitors paired with 100 nF and 10 nF ceramic decoupling capacitors across the logic rails.
- **Signal Conditioning:** 10 kΩ pull-up arrays, 4.7 kΩ I^2C terminators, 1 kΩ circuit-limiting resistors, and 330 Ω LED current limiters.

3 PCB Design and Manufacturing

3.1 Design for Manufacturing (DFM) and Layout Strategy

Geometric Track Clearance Solutions

The chosen manufacturer required a minimum 0.7 mm track-to-track clearance and 0.5 mm minimum annular rings. Standard 2.54 mm pitch component headers fail this geometric rule by default. To resolve this without manual tracing, a custom script dynamically modified the footprint library, optimizing pad definitions to 1.8 mm width with a 0.8 mm internal hole. This expanded layout clearances to a safe 0.74 mm, reducing the board's total DRC error count to zero.

Hybrid Routing Optimization

A hybrid routing approach was used to prevent clutter while maintaining a cost-effective two-layer stackup. Automated routing successfully traced 95% of standard low-frequency logic lines. Critical signal traces—power monitoring lines, sonar trigger paths, and high-speed UART lines—were manually routed on the top layer with explicit point-to-point jumpers.

Structural Branding Integration

Custom product identifier typography (*VMO*) was etched directly onto the top copper layer as isolated, floating conductive shapes surrounded by the ground pour. This provides an indelible branding element that satisfies all fabrication safety constraints without relying on silkscreen ink.

3.2 Visual Documentation and System Topology

The complete multi-tiered computing model and the data protocols linking the heterogeneous processors are mapped out in Figure 1. The mobile device forms a physical communication loop, acting simultaneously as the local wireless hotspot and the cloud uplink.

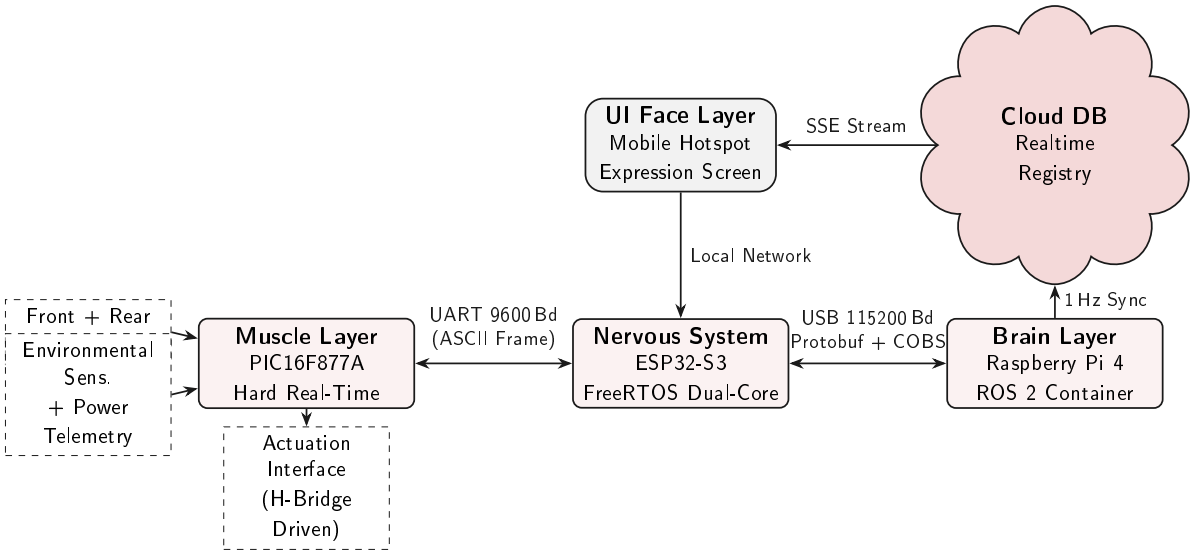

Figure 1: Distributed Heterogeneous Computing Hierarchy and Multi-Protocol Communication Network Topology.

The trace loops and ground pours in Figure 2 confirm zero DRC errors across the physical substrate. The fully populated 3D board view in Figure 3 confirms that all component footprints satisfy physical layout constraints.

4 Safety, Power Architecture, and Signal Integrity

4.1 Power Distribution Architecture

Primary power is sourced from a 3S Lithium-Polymer (LiPo) battery providing an 11.1 V nominal rail. The distribution chain follows a deeply layered safety hierarchy before reaching any logic device, as illustrated in Figure 4.

The distribution chain provides the following protection stages:

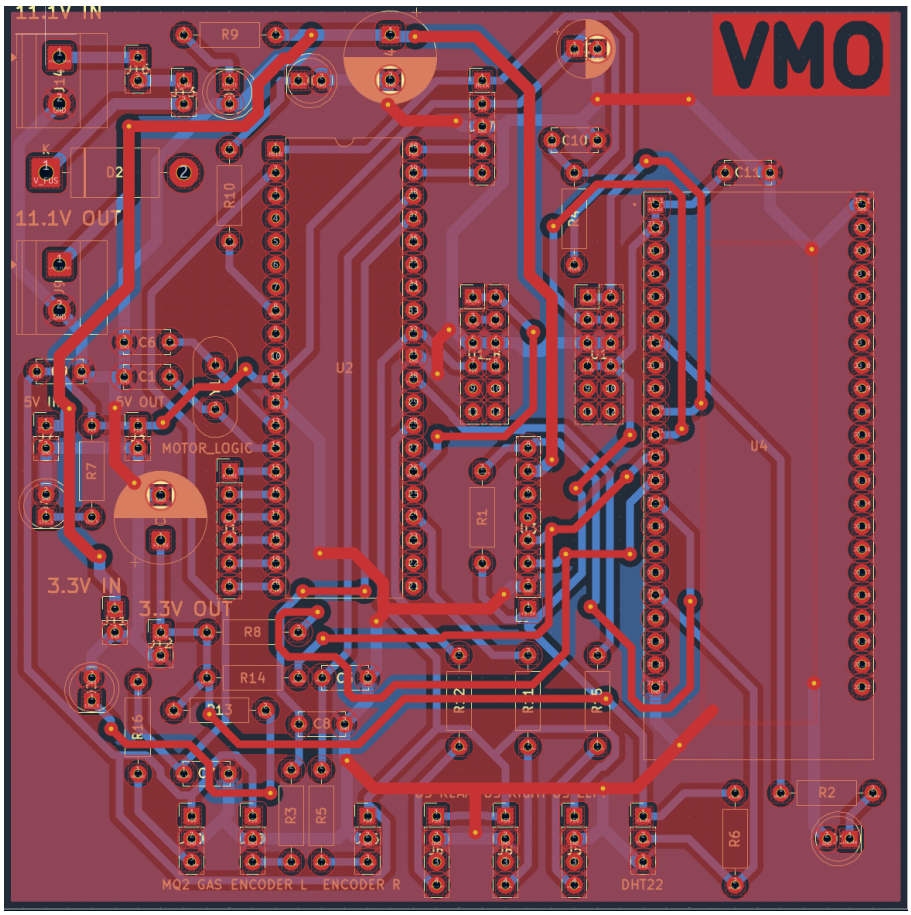

Figure 2: Two-Layer PCB Trace Routing and Ground Plane Layout.

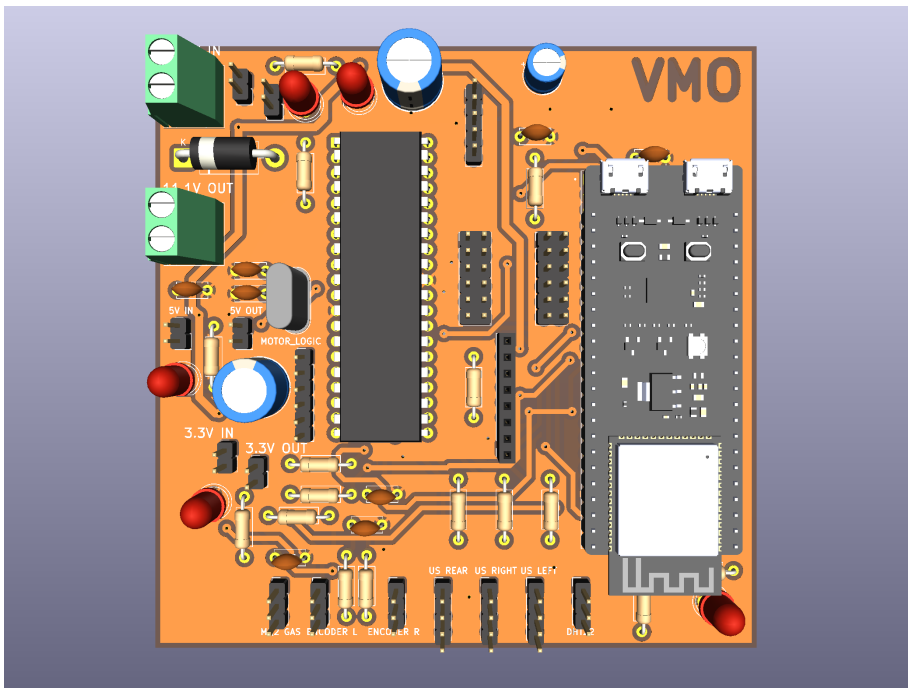

Figure 3: Complete 3D Render of the Populated VMO Baseboard.

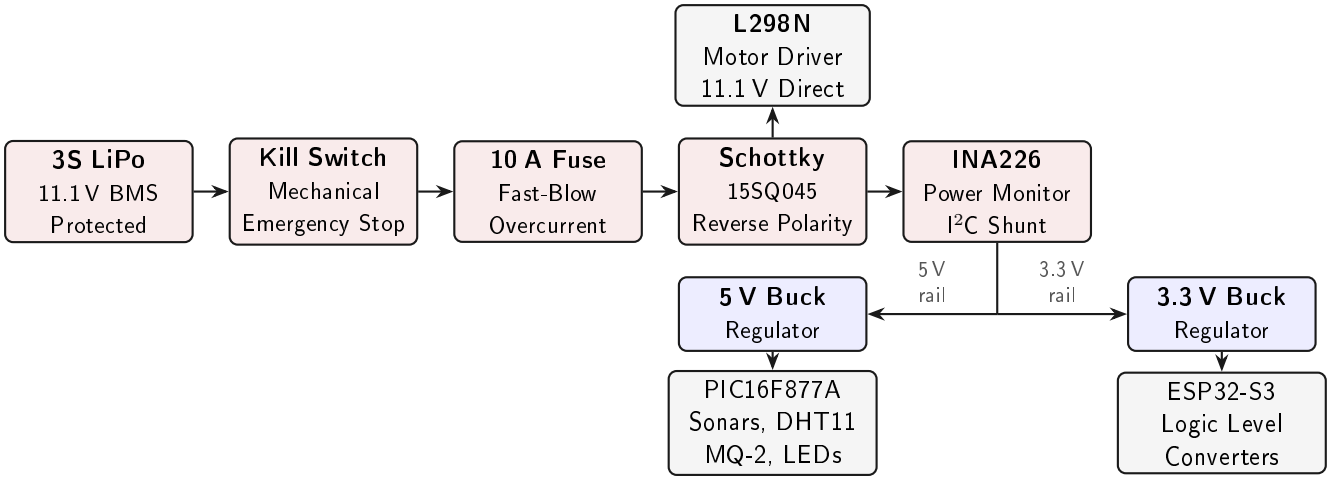

Figure 4: Power Distribution Chain and Dual-Rail Regulation Architecture.

- **BMS Protection Layer:** The 3S BMS module safeguards against over-discharge, over-charge, and short-circuit conditions at the cell level before any system-level protections.
- **Physical Interruption:** The mechanical kill switch breaks the 11.1 V feed before it reaches any regulator or driver, acting as a true hardware emergency stop.
- **Overcurrent Isolation:** A 10 A fast-blow fuse immediately downstream of the switch prevents lithium battery thermal runaway in any short-circuit event.
- **Reverse Polarity Protection:** The 15SQ045 Schottky diode guarantees zero current flows into downstream electronics if the main input is inadvertently cross-wired.
- **Active Power Monitoring:** The INA226 samples voltage and current over I^2C, enabling an automated software soft-shutdown protocol before battery degradation occurs.
- **Dual-Rail Regulation:** Two independent buck regulators provide clean, isolated 5 V and 3.3 V rails. The 5 V rail drives the PIC, sonar array, DHT11, MQ-2, and LEDs. The 3.3 V rail drives the ESP32-S3 and logic level converter low-side references. The L298N motor driver is powered directly from the post-diode 11.1 V bus to isolate inductive surges from logic rails.

4.2 Hardware Safety and Protection Features

In addition to the power architecture above, a set of I/O-level protections mitigates failures during physical operation.

- **I/O Short-Circuit Prevention:** 1 kΩ current-limiting resistors are integrated into the ultrasonic Echo signal routes, safeguarding the PIC if a software exception mis-initializes an input pin as a driven output.
- **Active Power Monitoring:** The INA226 continuously feeds power rail parameters to the processors over I^2C, enabling a software-level low-power soft shutdown before battery degradation occurs.

4.3 Anti-Noise and EMC/EMI Filtration

High-power DC motor drivers and wireless transceivers introduce severe electrical noise into shared copper rails. VMO implements strict geometric isolation on the PCB layout.

- **"Star Power" Grounding:** High-current motor supply returns trace directly back to the battery terminal blocks, never sharing a reference plane with the 5 V or 3.3 V logic rails. This prevents ground bounce from disrupting digital processing.
- **Dynamic Copper Ground Planes:** Large, uninterrupted top and bottom copper pours establish low-impedance return loops, acting as a shield against electromagnetic interference.
- **Targeted RF Shielding:** Ground copper pours are placed beneath the ESP32-S3 acting as both a thermal heatsink and an EMI shield. A distinct RF antenna keepout zone omits all copper layers around the integrated trace antenna to maximize wireless range.
- **Bulk and Local Decoupling:** 470 µF bulk capacitors cushion the wireless power rail against voltage droops during Wi-Fi transmission bursts. 100 nF surface-mount capacitors bypass high-frequency switching noise to ground at every IC power pin.
- **Hardware Signal Debouncing:** The gas sensor line and encoder interrupt paths include analog RC low-pass filters to suppress switch jitter and line ripple before data reaches the digital pins.

4.4 Low-Level Serial Frame Telemetry and Checksums

The PIC converts all raw analog metrics to whole integers and compiles them into a custom ASCII frame before transmission:

```
"«T[temp*10],H[hum*10],R[rear_cm],L[left_cm],X[right_cm],G[gas_adc]|CS>"
```

The terminal tag `CS` is an 8-bit XOR checksum computed across every character byte in the frame. The receiving task validates this checksum to guarantee that electrical line ripple cannot corrupt the operational state machines.

4.5 Asynchronous Task Scheduling and Protocol Transformation

The ESP32-S3 dual-core architecture distributes tasks deterministically via a prioritized FreeRTOS scheduler, decoupling the low-bandwidth PIC link from the high-speed Raspberry Pi uplink:

- `vTaskUARTParser` (Core 0, Priority 3): Drains the incoming serial feed, validates XOR checksums, and writes verified payloads behind a FreeRTOS mutex. A software watchdog flags communication loss if data drops for more than 3000 ms.
- `vTaskMotorCmd` (Core 0, Priority 4): Consumes directional commands from the thread-isolated queue `g_cmdQ` and outputs them to the motor controllers, enforcing a 100 ms post-send delay to eliminate FIFO buffer overruns.
- `vTaskPiComm` (Core 1, Priority 2): Governs high-speed transmission to the Raspberry Pi at 10 Hz over a native USB link.

To optimize bandwidth, the ESP32 drops text-based JSON frames in favor of a compact binary stream serialized with Protocol Buffers. Because binary data can contain null bytes that mimic newline delimiters, the stream is passed through a Consistent Overhead Byte Stuffing (COBS) encoder. This eliminates all raw null characters, allowing a clean `\x00` byte to serve as an unambiguous frame delimiter.

5 Schematic Analysis and Circuit Co-Simulation

5.1 Peripheral Netlist Assignment Matrix

The structural interconnect paths configured in the master CAD environment (`ES_project.pdsprj`) are cross-mapped to the hardware microcontroller execution layers in Table 1.

Peripheral Domain	Component	PIC Pin	Net Description
Primary Base Clock	X1 (20 MHz Crystal)	Pins 13, 14	Dual 22 pF load capacitors
Climate Telemetry	U3 (DHT22)	Pin 33 (RB0)	Single-wire, 4.7 kΩ pull-up
Toxic Gas Sensing	GAS1 (MQ-2)	Pin 2 (RA0)	10-bit internal ADC
Rear Distance Sonar	REAR (HC-SR04)	Pins 35, 36	TRIG_REAR→RB2; ECHO_REAR→RB3
Left Distance Sonar	LEFT (HC-SR04)	Pins 38, 39	TRIG_LEFT→RB5; ECHO_LEFT→RB6
Right Distance Sonar	RIGHT (HC-SR04)	Pins 40, 15	TRIG_RIGHT→RB7; ECHO_RIGHT→RC0
Diagnostic Visuals	D4 (Green LED)	Pin 34 (RB1)	Heartbeat via 330 Ω
Diagnostic Alerts	D5 (Red LED)	Pin 37 (RB4)	Fault indicator via 330 Ω
Actuation Driver A	U5 (L298 H-Bridge)	Pins 19–22	Direction inputs RD0–RD3
Actuation Driver B	U2 (L298 H-Bridge)	Pins 27–30	Direction inputs RD4–RD7

Table 1: Master Firmware Schematic Pin Registration and Netlist Interconnect Matrix.

5.2 Discrete MOSFET Logic Level Shifting Architecture

A critical challenge is interfacing the PIC16F877A's 5 V logic domain with the ESP32-S3's 3.3 V rails. Direct connection would cause electrical overstress on the ESP32 pins. The schematic uses discrete `2N7002` N-channel MOSFETs (`Q1`, `Q2`) with 10 kΩ pull-up networks (`R7`–`R10`).

Mathematical Model of Transistor-Level Voltage Handoff

Three distinct operating states govern the level-shifting transceiver:

1. **Steady-State High Idle:** Both nodes idle at logic high. The gate-to-source voltage $V_{gs} = 0\,V$ places the MOSFET in cut-off. Both lines are independently pulled to their respective rails via `R9` (3.3 V) and `R10` (5 V).

2. **Low-Side Drive (3.3 V Driving Low):** The ESP32 pulls the 3.3 V node to 0 V:

$$V_{gs} = V_{Gate} - V_{Source} = 3.3\,V - 0\,V = 3.3\,V > V_{gs(th)} \approx 2.1\,V$$

   The channel saturates, conducting through the open drain and pulling the 5 V PIC node to 0 V.

3. **High-Side Drive (5 V Driving Low):** The PIC pulls the 5 V node to 0 V. The internal parasitic body diode becomes forward-biased, clamping the 3.3 V node to:

$$V_{Low} \approx 0.7\,V$$

   This pulls the MOSFET source below gate potential, triggering $V_{gs} \approx 2.6\,V > V_{gs(th)}$. The channel fully opens, driving the 3.3 V node to clean 0 V.

5.3 Multi-Bridge Actuation Topology

The schematic routes control signals to two L298 motor driver blocks (`U2` and `U5`). Four individual lines across `PORTD` are dedicated to each H-bridge module, providing the hardware infrastructure required for skid-steer and four-wheel kinematic modes. This multi-bridge layout buffers high-current inductive surges up to 12 V, completely isolating the logic chips from actuation currents.

5.4 CAD Schematic Layout

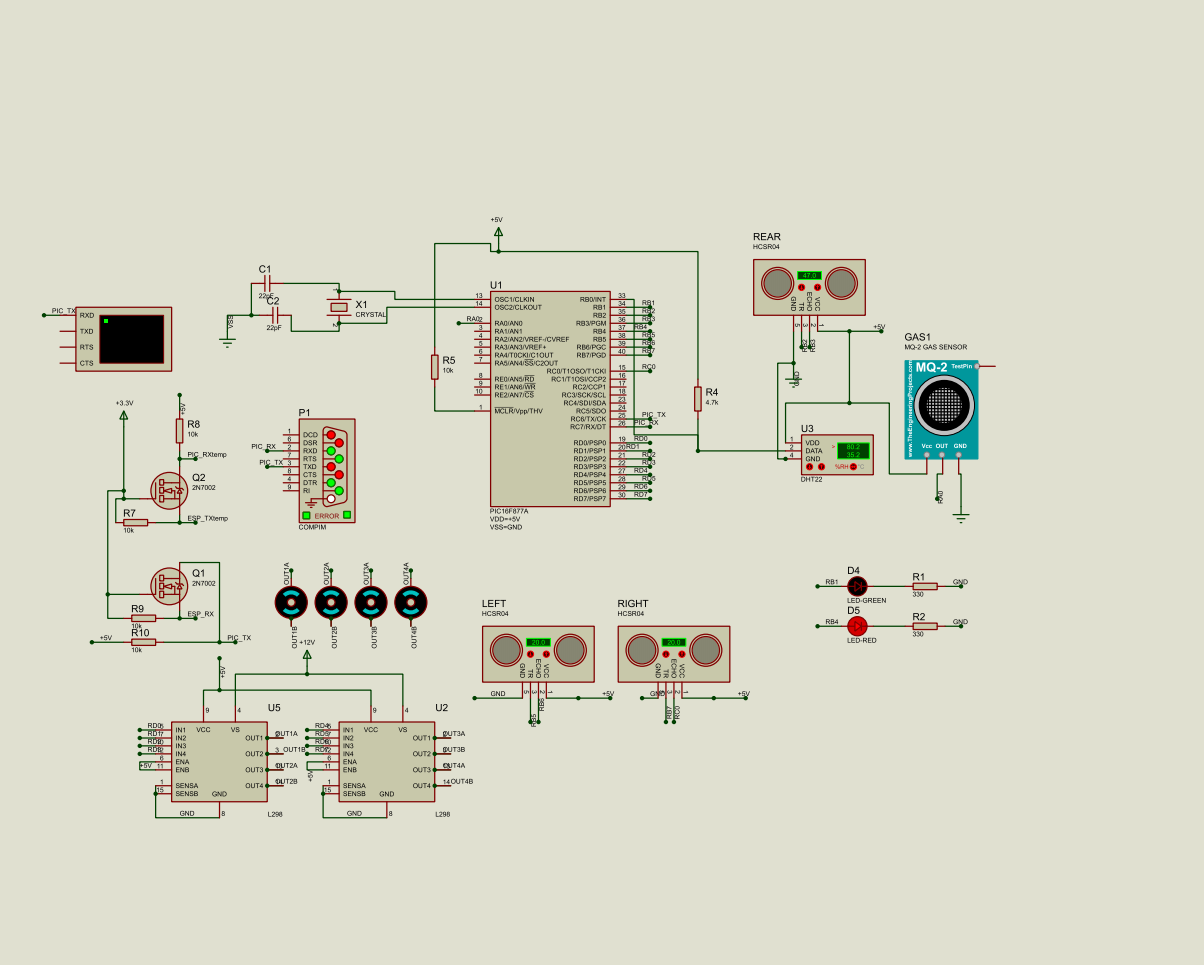

Figure 5: Complete Electrical Interconnect Schematic and Peripheral Communication Network Model.

Figure 5 confirms that all simulation configurations pass electrical verification. Low-power communication lines are cleanly separated from high-power inductive motor returns, satisfying all spatial trace routing safety boundaries.

6 Hard Real-Time Firmware Infrastructure

6.1 Layered Architecture and Split Responsibility Paradigm

The VMO firmware for the PIC16F877A is segregated into four isolated layers, as illustrated in Figure 6.

A strict split-responsibility paradigm is enforced across computing nodes. All high-level navigation choices, safety overrides, speed ramping profiles, and threshold evaluations are offloaded to the upstream ESP32-S3 and Raspberry Pi 4. The PIC is kept as a deterministic actuator with only two roles: reading physical sensor inputs and executing raw output commands as directed by the ESP32 over a fixed 9600 Baud UART link. This ensures that changes to navigation logic never require re-flashing the PIC firmware.

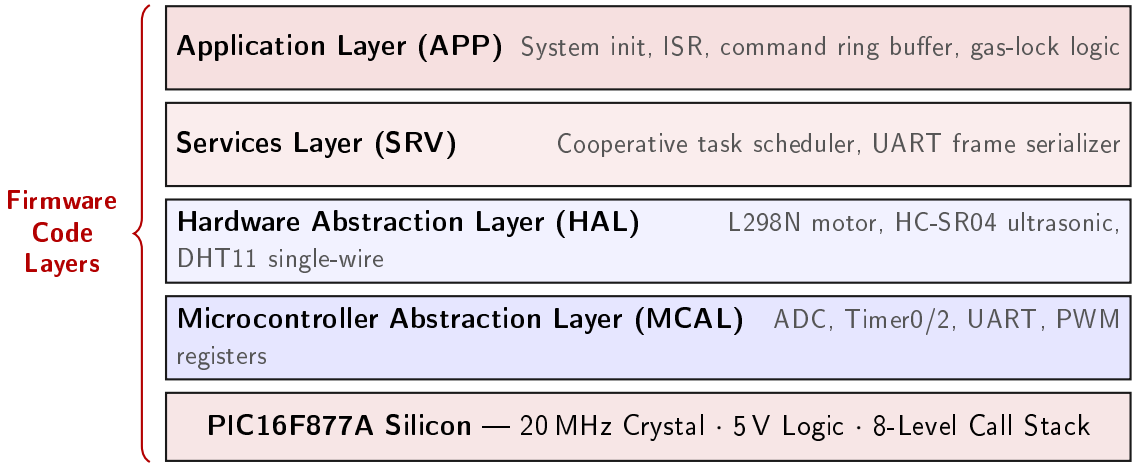

Figure 6: PIC16F877A Firmware Layered Software Architecture.

6.2 Microcontroller Abstraction Layer (MCAL) Configuration

The MCAL programs the PIC's internal peripherals using timing metrics derived from the 20 MHz external crystal. The internal instruction execution frequency ($F_{osc}/4$) is a stable 5 MHz, providing a base cycle resolution of 0.2 µs.

Timer0 System Tick Generation

The global system clock counter (`g_tick_ms`) is generated by an 8-bit Timer0 configured with a 1:32 prescaler, incrementing the register every 6.4 µs. The required count per 1 ms overflow is:

$$\text{Required Counts} = \frac{1\,\text{ms}}{6.4\,\mu\text{s}} = 156.25 \approx 156$$

The Timer0 preload is set to $256 - 156 = 100$. On every overflow, the ISR reloads the register to 100 and increments the tracking parameter, maintaining a stable 1 ms timeline.

Atomic Tick Reading Logic

Reading the 32-bit variable `g_tick_ms` on an 8-bit architecture requires four sequential byte-read operations. A Timer0 interrupt firing mid-read would corrupt the result. The function `MCAL_GetTick()` suspends the Global Interrupt Enable bit (`GIE`), performs an atomic copy, and immediately restores the interrupt state.

UART Sub-system Configuration

The hardware UART uses high-speed asynchronous mode (`BRGH = 1`). The baud rate register is calculated as:

$$\text{SPBRG} = \left(\frac{F_{osc}}{16 \times \text{Baud}}\right) - 1 = \left(\frac{20{,}000{,}000}{16 \times 9600}\right) - 1 = 129.21 \approx 129$$

Loading `SPBRG` with 129 yields 9615 Baud — a 0.16% error margin, well within stable synchronization limits with the ESP32.

ADC and PWM Configurations

The 10-bit ADC maps exclusively to pin `AN0/RA0` for the MQ-2 sensor, right-justified in `ADRESH:ADRESL` with $F_{osc}/32$ conversion clock to satisfy acquisition timing requirements.

PWM motor actuation uses the CCP1 and CCP2 hardware modules. Timer2 runs with a 1:16 prescaler and PR2 = 155, generating a stable 2 kHz PWM frequency. The upper 8 duty-cycle bits route to CCPRxL; the lower 2 bits are packed into bits [5:4] of CCPxCON.

6.3 Hardware Abstraction Layer (HAL) Driver Development

L298N Motor Driver

VMO's locomotion uses an L298N dual H-bridge:

- **Left Axis:** Direction pins IN1 (RD0) and IN2 (RD1); speed via CCP2 on RC1.
- **Right Axis:** Direction pins IN3 (RC0) and IN4 (RC3); speed via CCP1 on RC2.

Direction changes modify PORTC through an isolated shadow variable s_portc, preventing read-modify-write conflicts from corrupting adjacent active PWM signals on RC1/RC2. Standard directional travel uses duty cycle 600; in-place turns use 400.

Ultrasonic Sensor Timeout Optimization

The HC-SR04 driver fires a 10 µs trigger on TRIG_REAR (RB2), then captures the echo width on ECHO_REAR (RB3) using Timer1 at 2.5 MHz (0.4 µs/tick). Distance is calculated as:

$$\text{Distance (cm)} = \frac{\text{Timer1 Ticks}}{145}$$

During integration, a critical bug was found: the echo-polling watchdog was set to 30,000 µs, blocking the main loop for up to 30 ms when facing open space. This filled the 2-byte UART FIFO, dropping motor commands from the ESP32. The timeout was reduced to 5,000 µs — sufficient to capture all valid reflections while eliminating the blocking stall. Unresponsive paths return DIST_OUT_OF_RANGE = 999.

DHT11 Single-Wire Protocol

The DHT11 requires microsecond-level timing accuracy. The driver disables global interrupts via GIE for the duration of the single-wire transaction to prevent background task interference. The protocol pulls the line low for 18 ms, releases to input for 30 µs to read the acknowledgment pulse, then extracts a 40-bit data packet. Each bit is evaluated by a low-to-high transition followed by a 40 µs delay: pin high = logical 1, pin low = logical 0. Results are multiplied by 10 to preserve one decimal place as an integer, avoiding floating-point overhead.

6.4 Services Layer and Cooperative Task Scheduling

The PIC16F877A cannot support a preemptive RTOS — context switching would exhaust its 8-level hardware call stack. Instead, the services layer implements a lightweight non-preemptive cooperative time-based task dispatcher.

The scheduler uses a fixed 4-slot descriptor matrix recording function pointers, intervals, enabled flags, and last-run timestamps. SRV_Scheduler_Run() executes in the main loop, reads the atomic tick once per cycle, and dispatches any task whose period has elapsed. The four registered tasks are shown in Table 2.

Task	Period	Function
task_adc	100 ms	Reads MQ-2 ADC; evaluates gas safety lock flag (3 consecutive exceedances of ADC $>$ 400 set s_gas_locked).
task_ultrasonic	200 ms	Triggers rear HC-SR04 sonar distance measurement.
task_dht11	2000 ms	Climate update; retains previous readings if checksum validation fails.
task_uart_tx	250 ms	Serializes and broadcasts the ASCII telemetry frame with XOR checksum.

Table 2: Cooperative Task Scheduler: Registered Tasks, Periods, and Responsibilities.

Stack-Safe String Serialization

The transmitted ASCII frame takes the form:

"«T{temp},H{hum},R{rear_dist},L0,X0,G{gas}|{XX}>\r\n"

The serialization layer deliberately avoids sprintf. The XC8 toolchain's sprintf uses recursive paths that consume multiple levels of the 8-level call stack. Instead, a custom digit-reversal loop converts integers to ASCII via repeated modulo-10 operations and integer division, guaranteeing zero stack overflow risk.

6.5 Application Layer and Interrupt Command Ring Architecture

Peripheral Initialization

Key configuration directives:

- FOSC = HS: High-speed external crystal oscillator mode.
- WDTE = OFF: Hardware watchdog disabled during development.
- LVP = OFF: Low-voltage programming disabled, freeing RB3 for the rear sonar echo input.

Port directions are set via TRIS register masks:

Register	Mask	Assignment
TRISB	0b00001001	Inputs: RB0 (DHT11), RB3 (Echo); outputs: all others.
TRISC	0b10000000	Input: RC7 (UART RX); outputs: motor direction and PWM.
TRISD	0x00	Full PORTD as digital outputs.
TRISA	0xFF	Full PORTA as analog inputs.

Interrupt-Driven Command Ring Buffer

Because the main loop is occupied by sensor polling, incoming steering commands from the ESP32 are handled via an interrupt-driven circular ring buffer (s_cmd_buf) with 8 slots governed by read/write head pointers.

On RCIF interrupt, the ISR handles three hardware exception cases:

1. **Overrun (OERR flag):** Flushes both FIFO bytes, toggles CREN to restart the receiver, and asserts s_dbg_oerr.
2. **Framing Error (FERR flag):** Drops the corrupted byte and asserts s_dbg_ferr.
3. **Normal Byte:** If the character matches one of 'F', 'B', 'L', 'R', 'S', it is pushed to the ring buffer. Unknown bytes are discarded and s_dbg_ign is asserted.

Command Execution and Deduplication

At the top of each main cycle, the ring buffer is drained and commands dispatched to the motor functions. A deduplication check skips any character matching the previously executed state, preventing redundant H-bridge calls when the ESP32 transmits held movement commands at high frequency:

`'F'` → Forward `'B'` → Backward `'L'` → Left Turn `'R'` → Right Turn `'S'` → Stop

Per the split-responsibility paradigm, the PIC firmware entirely ignores the gas lock flag `s_gas_locked` during command execution. The decision to override motor states during a hazard alarm is delegated exclusively to the ESP32 software layer, keeping PIC firmware stable and maintenance-free once flashed.

7 Nervous System Architecture and Edge Middleware

7.1 FreeRTOS Dual-Core Task Distribution

The ESP32-S3 runs a deterministic FreeRTOS architecture, balancing asymmetric workloads across dual Xtensa LX7 cores. Execution blocks are pinned to specific cores and isolated by priority:

- `vTaskMotorCmd` (Core 0, Priority 4 — Highest): Un-pools character-mapped drive codes from the 16-depth queue `g_cmdQ` and translates them to pin states with zero-latency processing.
- `vTaskSensorReader` (Core 0, Priority 3): Polls the tri-directional sonar array, DHT22 climate sensor, and MQ-2 gas channel on a structured loop.
- `vTaskPiComm` (Core 1, Priority 2): Serializes sensor data into Protobuf+COBS binary frames and transmits over the native USB link to the Raspberry Pi.
- `vTaskWebServer` (Core 1, Priority 2): Services asynchronous HTTP client requests, streaming telemetry and decoding cockpit drive commands.
- `vTaskWiFiGuard` (Core 1, Priority 1 — Lowest): Monitors the Wi-Fi link every 5000 ms, automatically disconnecting, flushing stale stack allocations, and re-establishing the connection on dropout.

7.2 ESP32 Core v3.x PWM API and Speed Mapping

The firmware uses the updated ESP32 Arduino Core v3.x API, replacing the legacy `ledcSetup()/ledcAttachPin()` channel-assignment calls that introduced hardware dependency issues. The new single-call interface:

```
ledcAttach(PIN_L298N_ENA, 5000, 8);
```

provisions a 5 kHz PWM signal at 8-bit resolution (0–255 steps) directly on the target pin.

Speed values are transmitted as ASCII characters `'0'` through `'9'`. A linear mapping translates the character index into a raw PWM duty cycle value:

$$\texttt{g\_speed\_raw} = \text{round}((\texttt{character} - \texttt{'0'}) \times 28.33)$$

The duty cycle is written to the H-bridge via:

```
ledcWrite(PIN_L298N_ENA, g_current_speed_raw);
```

7.3 Asynchronous HTML5 Cockpit Dashboard and Bitwise Motor Mapping

The middleware hosts an asynchronous HTTP cockpit interface on port 80, built in vanilla HTML5/CSS3/JavaScript for lightweight rendering on mobile screens.

Dynamic Motor Re-mapping

The interface stores a four-channel H-bridge direction matrix in browser `localStorage`, allowing field engineers to remap motor wiring on-the-fly from the dashboard — swapping logic levels across `IN1`–`IN4` — without flashing either microcontroller.

Hexadecimal Bitwise Command Encoding

To prevent the Wi-Fi link from being saturated during rapid driving, direction states are packed into a single character. On button press, the four Boolean pin states are packed into a 4-bit nibble:

$$\text{Value} = (\text{IN1} \ll 3) \mid (\text{IN2} \ll 2) \mid (\text{IN3} \ll 1) \mid \text{IN4}$$

$$\text{CharCode} = \texttt{String.fromCharCode}(97 + \text{Value})$$

This maps to a single character between `'a'` and `'p'`. The ESP32 extracts the pin states via bitwise masking:

$$\text{IN1} = (\text{val} \gg 3)\,\&\,0x01 \quad \text{IN2} = (\text{val} \gg 2)\,\&\,0x01 \quad \text{IN3} = (\text{val} \gg 1)\,\&\,0x01 \quad \text{IN4} = \text{val}\,\&\,0x01$$

7.4 Acoustic Isolation and Sensor Acquisition Filters

The `vTaskSensorReader` loop uses a FreeRTOS mutex (`g_mutex`) to protect sensor data structures from race conditions during concurrent web or Pi communication reads.

Cross-Talk Minimization via Phase Delays

Running multiple ultrasonic transducers simultaneously introduces acoustic interference — reflections from one sensor can trip adjacent echo pins. The acquisition loop triggers each transducer sequentially, inserting an explicit 15 ms decay window between channels:

```
vTaskDelay(pdMS_TO_TICKS(15));
```

This pause allows stray acoustic energy to fully dissipate before the next sonar fires, ensuring clean distance data across all three quadrants.

Environmental Multi-Sample Scaling

Temperature, humidity, and gas levels change slowly and bypass the 500 ms sonar loop. Using a local iteration counter, environmental channels are sampled exactly once every fourth sonar cycle (2000 ms). Sensor failures or timeouts increment `err_count` and log to `last_err` for engineering diagnostics.

8 Containerized Runtime Environment

8.1 Virtualization Strategy and Dependency Isolation

The entire cognitive framework of VMO runs inside an isolated headless Linux Docker container. This eliminates dependency conflicts on the host OS, ensuring identical compiler optimizations, Python library stacks, and ROS 2 workspaces execute cleanly regardless of developer workstation configuration.

Container parameters:

- **Image Name:** `mobile-robot-base:v1`
- **Base Stack:** Ubuntu 22.04 LTS (Jammy Jellyfish) for ARM64, with ROS 2 Humble base.
- **Container Name:** `robot_env`
- **Volume Mapping:** Host path `/home/mattmatt/robot_ws` (Pi) or `D:\ES_Project\robot_ws` (laptop) is mounted to `/root/robot_ws`, enabling live source-code editing over SSH while compilation runs natively inside the container.

8.2 Production Dockerfile Configuration

Dockerfile — Production Environment Blueprint

```
FROM ros:humble-ros-base-jammy
ENV DEBIAN_FRONTEND=noninteractive

# 1. System packages and build tools
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential cmake git curl nano python3-pip \
    python3-colcon-common-extensions v4l-utils \
    libopencv-dev protobuf-compiler \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

# 2. Python robotics and cloud dependencies
RUN pip3 install --no-cache-dir \
    pyserial cobs protobuf firebase-admin numpy opencv-python-headless

# 3. Project workspace structure
WORKDIR /root/robot_ws
RUN mkdir -p /root/robot_ws/src/pet_brain/protos

# 4. ROS 2 environment sourcing
RUN echo "source /opt/ros/humble/setup.bash" >> /root/.bashrc

# 5. Communication ports
EXPOSE 5000/udp 5001/udp

ENTRYPOINT ["/ros_entrypoint.sh"]
CMD ["bash"]
```

Table 3: Containerized ROS 2 Stack Deployment Blueprint.

8.3 Deployment and Hardware Device Passthrough

Build Command

```
docker build -t mobile-robot-base:v1 .
```

Secure Runtime Launch

```
docker run -it \
    --name robot_env \
    --net=host \
    --privileged \
    -v /home/mattmatt/robot_ws:/root/robot_ws \
    --device=/dev/video0:/dev/video0 \
    --device=/dev/ttyUSB0:/dev/ttyUSB0 \
    mobile-robot-base:v1
```

The runtime flags provide two key architectural safeguards:

1. **Dependency Isolation:** All build tools and Python packages remain inside the container layer, leaving the host OS unaltered.
2. **Granular Hardware Passthrough:** Explicit `-device` flags open narrow, policed conduits to the webcam (`/dev/video0`) and the ESP32 USB link (`/dev/ttyUSB0`) without exposing the entire host peripheral bus.

9 Edge-AI and Machine Learning Perception Engine

9.1 Hardware-Optimized Memory Sandbox

To prevent standard ML runtimes (PyTorch, Pandas) from triggering an out-of-memory kernel panic on the 2 GB Raspberry Pi, the VMO vision tracking system is written entirely in native C++. It interfaces directly with the official pre-compiled ARM64 ONNX Runtime C++ API binaries. Compiler flag `-O3` maximizes loop unrolling and CPU pipeline optimization; build memory is controlled by single-threaded compilation via `make -j1`.

9.2 Multi-Threaded Machine Vision Architecture

The executable splits perception into three isolated POSIX threads:

1. **Producer Thread:** Ingests frames from a USB webcam at 1280×720 at 30 FPS, with an automated fallback scanner cycling through `/dev/video0`–`/dev/video5` on device collision. Acquired frames are pushed to a thread-safe queue capped at 2 to eliminate accumulation lag.
2. **Consumer Thread:** Runs the ONNX inference model on dequeued frames, converts outputs to bounding-box tracking coordinates, and fires them over a local loopback UDP socket to the ROS 2 graph on port 5001.
3. **Display Thread:** Pulls frames from the processing queue, applies bounding-box overlays, and downsamples the feed to a compressed JPEG stream for real-time wireless debugging without impacting inference performance.

9.3 YOLO26 Nano Object Detection

The vision core uses a YOLO26 Nano deep convolutional network (`yolo26n.onnx`), optimized for edge inference.

NMS-Free Architecture

Standard detectors require a serial Non-Maximum Suppression (NMS) loop on the CPU to filter redundant bounding boxes — a bottleneck that scales poorly on low-power hardware. YOLO26 Nano eliminates this by using an end-to-end dual-assignment architecture that produces one-to-one object mappings natively within the network graph. Bounding box coordinates for Class 0 (Person) are parsed directly from the ONNX Runtime output tensor, bypassing any CPU post-processing.

Input Tensor Normalization

The inference runtime captures each raw frame, converts it to a normalized float array at 640×640 pixels, applies channel-wise mean and standard deviation scaling, and injects it into the execution graph.

9.4 Unique Frames Processing Algorithm

To keep the Raspberry Pi 4 below its thermal throttling boundary ($> 70°$C), a pre-inference filter called the **Unique Frames Processing Algorithm** intercepts incoming frames before tensor allocation.

Each incoming frame F_t is converted to grayscale and downsampled via bilinear interpolation to a 64×64 snapshot matrix s_t. The absolute mean pixel deviation δ is computed against the last verified unique snapshot U_{last} over $N = 4096$ pixels:

$$\delta = \frac{1}{N} \sum_{i=1}^{N} |s_t(i) - U_{\text{last}}(i)|$$

This is evaluated against a calibrated threshold $\tau = 4.5$:

1. **Static State ($\delta \leq \tau$):** Frame is flagged as a duplicate. The buffer is released immediately, bypassing all tensor creation, ONNX evaluation, and memory writes.
2. **Unique Motion State ($\delta > \tau$):** Frame is flagged as containing new spatial information. The reference cache is updated ($U_{\text{last}} \leftarrow s_t$) and the full-resolution frame is forwarded to the YOLO26 Nano inference core.

By skipping inference on static scenes, this algorithm reduces unnecessary CPU load, maintains an optimal thermal envelope, and eliminates frame backlog lag during extended runtimes.

10 ROS 2 Control Topology and Kinematics Engine

10.1 Distributed ROS 2 Microservices Pipeline

Rather than embedding computer vision dependencies inside a standard ROS node, VMO uses a decoupled microservices approach. The standalone C++ vision core streams detection data over local UDP sockets to a centralized Python ROS 2 graph:

- **Hardware Serial Bridge (`serial_bridge.py`):** Streams at 115200 Baud, unpacks COBS framing, and publishes sensor data as JSON to the `/esp_sensors` topic. Outbound motion vectors are re-serialized into Protobuf before transmission to the ESP32.

- **Cloud Telemetry Node (`firebase_bridge.py`):** Subscribes to sensor and motor state topics, accumulates logs locally, and syncs to Firebase exactly once per second (1 Hz) to avoid bandwidth waste. Manages a deterministic emotion state machine: `"SCARED"` when safety distances are violated, `"HAPPY"` during active human pursuit, and `"SLEEPY"` when velocity commands have timed out for more than 3.0 s.

10.2 Subsumption Architecture Behavioral Priority Mapping

Navigation decisions are computed in `navigator.py` using a strict Subsumption Control Model, as diagrammed in Figure 7. Critical hazard reflexes sit above high-level navigation vectors and override them when triggered.

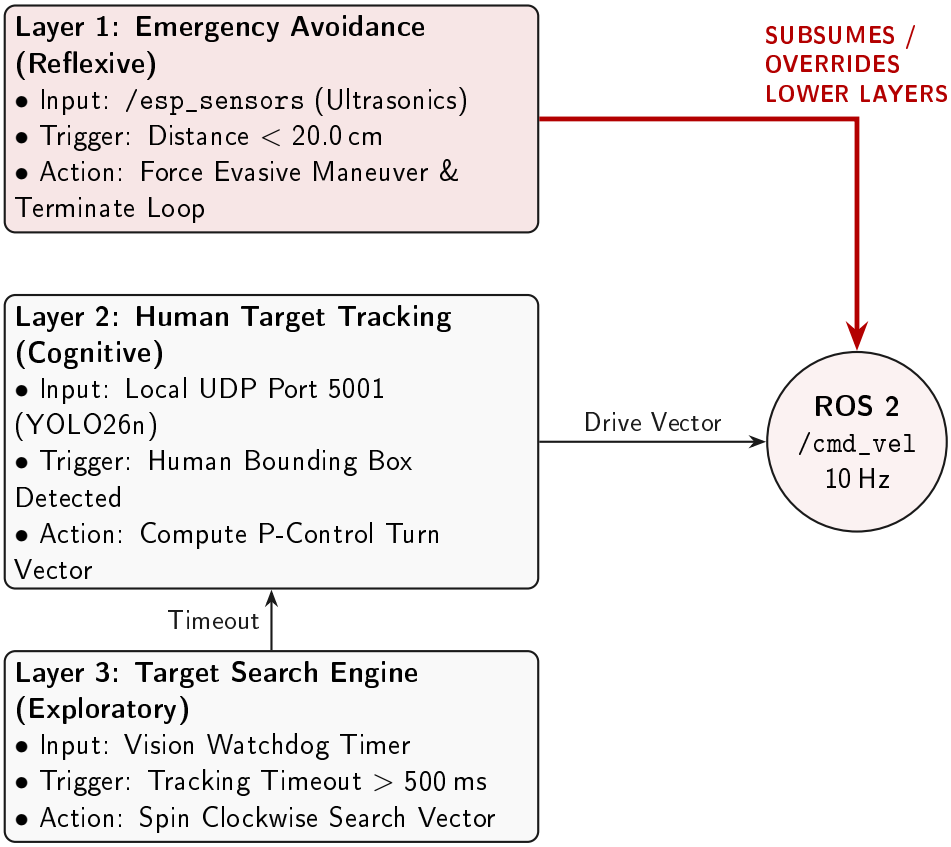

Figure 7: Subsumption Control Model: Layered Priority Tree with Emergency Reflex Override.

The layered model runs on every 10 Hz tick. **Layer 1** fires when any front ultrasonic distance drops below 20.0 cm. It immediately commands reverse velocity and an angular escape vector directed away from the nearest wall, then returns early to truncate lower-layer execution.

If the safety zone is clear, **Layer 2** engages Proportional Control tracking. The cross-track error $e_x(t)$ relative to the camera midpoint is:

$$e_x(t) = X_{\text{center}} - X_{\text{target}}$$

The corrective angular velocity command is:

$$u_{\text{angular}}(t) = K_p \cdot e_x(t), \quad K_p = 0.003$$

Linear velocity is regulated by the detected bounding box width w: a forward velocity of $0.2\,\text{m/s}$ is commanded when $w < 150$ to close the gap; the robot backs away at $-0.1\,\text{m/s}$ when $w > 250$ to preserve personal interaction space.

If no detections arrive within a 500 ms watchdog window, the system falls to **Layer 3**, spinning in place at $\omega_z = 0.3$ rad/s until the target re-enters frame.

10.3 Sensor Fusion and Graceful Kinematic Degradation

The navigation node reads the configuration variable `num_encoders` at startup to select its kinematic model:

1. **Open-Loop (`num_encoders` $= 0$):** Estimates displacement from power level and actuation duration.
2. **Differential Drive (`num_encoders` $= 2$):** Calculates linear and angular velocity from wheel speeds with track width L:

$$v = \frac{v_{\text{right}} + v_{\text{left}}}{2}, \qquad \omega = \frac{v_{\text{right}} - v_{\text{left}}}{L}$$

3. **Four-Wheel Skid-Steer (`num_encoders` $= 4$):** Averages velocities across all four wheels to filter out slip errors during high-speed pivot turns.

Data-Age Hardware Watchdogs

The localization pipeline validates data freshness on every wheel tick. The age of each incoming packet is evaluated against the system clock:

$$\Delta t = t_{\text{current}} - t_{\text{packet\_arrival}}$$

If Δt exceeds 100 ms, the engine logs a hardware fault warning, drops the stale metrics, and falls back to open-loop dead reckoning, ensuring navigation continues without halting system processes.

11 Mobile Core Intelligence and Frontend

11.1 Cost Optimization and Hardware Abstraction

The VMO platform uses a consumer Android 14+ smartphone as an integrated hardware abstraction layer. The smartphone's high-resolution display, multi-microphone array, audio drivers, network antennas, and processing core collectively serve as VMO's face, microphone, and cloud gateway. This eliminates separate procurement of an OLED display module, audio amplifier, speaker driver, and dedicated voice capture ADC during the prototype phase.

11.2 Dual-Stage Speech Processing

Speech interaction is managed through a hardware-safe, dual-stage pipeline illustrated in Figure 8.

Offline Wake-Word Detection

The Vosk Offline Android SDK links to a native C++ acoustic library via JNI, sampling audio at 16,000 Hz and matching it against a local voice-print tuned for the keyword `"Veemo"`. This background process runs entirely on local CPU registers, maintaining data privacy and minimal idle power draw.

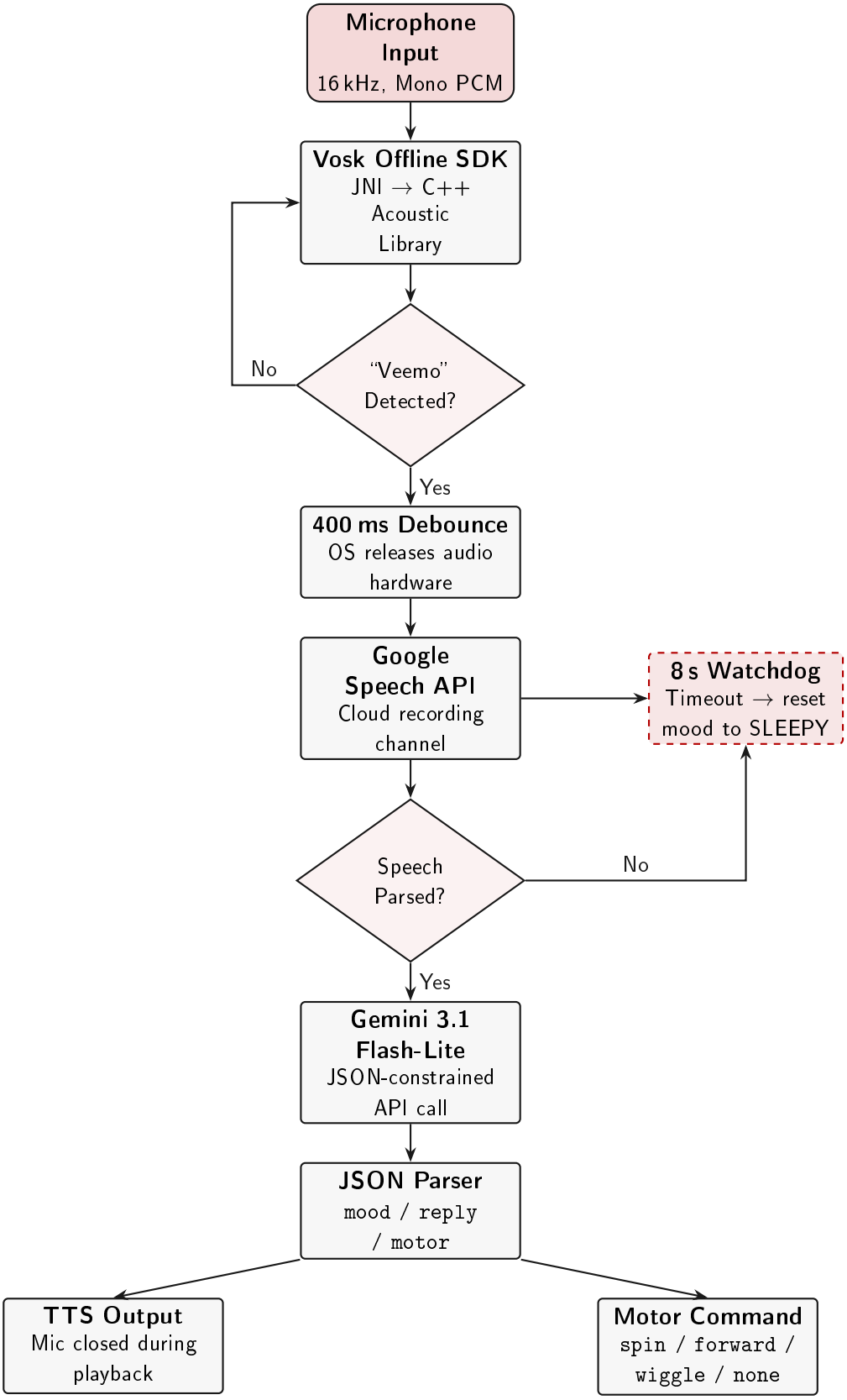

Figure 8: Mobile Speech Processing and LLM Integration Pipeline.

Hardware-Debounced Cloud Handoff

On a wake-word match, an explicit 400 ms delay is enforced before the cloud speech channel is initialized. This window allows the Android OS to fully release the audio hardware, preventing microphone allocation deadlocks between the Vosk background process and the Google Speech Recognition API.

Asynchronous Watchdog Timers

A main-thread watchdog timer (`watchdogRunnable`) manages two cascading fallbacks: if the cloud API fails to initialize, the system purges the frozen instance and restarts offline monitoring; if speech is captured but the pipeline fails to return text within 8 s, the service is hard-reset and the mood state is defaulted to `"SLEEPY"`.

11.3 JSON-Enforced Agentic Cognitive Architecture

The parsed speech text is dispatched to the Gemini 3.1 Flash-Lite API. System prompt constraints force the model to return data exclusively within a structured JSON object:

```
{ "mood": "[ENUM]", "reply": "[STRING]", "motor": "[STRING]" }
```

- `"mood"`: Governs the global behavioral state machine; restricted to `HAPPY`, `SCARED`, `SLEEPY`, `ANGRY`, or `SAD`.
- `"reply"`: Short conversational response, one to two sentences, passed to the TTS engine.
- `"motor"`: Discrete actuation directive sent to the chassis controllers — `spin`, `forward`, `wiggle`, or `none`.

A dedicated parsing layer strips any residual Markdown wrappers from the model's response (e.g., ```` ```json ````), converts the clean string to a native object, and distributes the parameters to the appropriate downstream systems.

11.4 Locked 60 Hz Vector Rendering and Animation

VMO's graphical expression system runs on a Jetpack Compose Canvas framework at a locked 60 Hz render rate.

Figure-8 Idle Drift Animation

To keep the robot appearing lifelike while idle, eye center coordinates $(X_{\text{eye}}, Y_{\text{eye}})$ are continuously modulated by a low-frequency figure-8 function:

$$X_{\text{eye}}(t) = X_{\text{center}} + 12 \cdot \sin(t)$$

$$Y_{\text{eye}}(t) = Y_{\text{center}} + 8 \cdot \cos(0.5 \cdot t)$$

This drift is overlaid with state-specific motion: high-frequency jitter in `SCARED` mode, or an orbital path at $\omega_{\text{orbit}} = 4.19$ rad/s during a `THINKING` evaluation loop.

Spring-Physics Transitions

Emotional state transitions—eye scale changes, eyebrow angle shifts—use native spring-physics animations (`Spring.DampingRatioMediumBouncy`, `Spring.StiffnessLow`) rather than brittle linear interpolation, producing organic movement profiles.

Synchronized TTS and Microphone Gating

Vocal output uses an offline TTS engine at $1.8\times$ pitch scale and $1.1\times$ speech rate. The microphone channel is gated closed during TTS playback to prevent VMO from capturing its own voice. The exact instant the TTS completion packet (`onDone`) fires, a main-thread handler re-activates the offline wake-word engine.

11.5 Split-Pipe Edge Routing and Cloud Telemetry

The application operates two parallel network pipelines over the local hotspot:

1. **Edge Real-Time Loop:** Cognitive state updates and motor commands are sent directly to the Raspberry Pi 4 container IP over the local subnet, achieving sub-5 ms latency to guarantee agile reflex loops during active pursuit.
2. **Cloud Analytics Pipeline:** Out of band from the motor loop, all tracking metrics, speech strings, conversation histories, and system logs are asynchronously pushed to the remote Firebase instance (`vmo-pet-default-rtdb.europe-west1.firebasedatabase.app`), providing long-term storage and diagnostics without adding latency to physical reflexes.

12 Testing, Validation, and Iterative Improvements

During the physical integration and dynamic testing phases of the VMO prototype, several hardware and processing bottlenecks were identified under real-world operating conditions. These challenges prompted iterative architectural improvements that significantly stabilized the final platform.

12.1 Thermal Bottlenecks and the Unique Frames Solution

Initial testing of the standard YOLO26 Nano model on the Raspberry Pi 4 yielded a suboptimal frame rate averaging 18 FPS. Furthermore, continuous tensor allocation on 30 incoming frames per second pushed the Pi's CPU array to its thermal throttling boundary ($> 70^{\circ}$C). Under sustained operation, this induced severe queue backlogs and motion-tracking lag.

To resolve this without resorting to active cooling hardware, the *Unique Frames Processing Algorithm* was conceptually designed and injected into the pipeline as a pre-inference filter. By evaluating mean pixel deviations on downscaled 64×64 matrices, the system successfully identified and bypassed completely static frames. This software-level intervention slashed unnecessary CPU overhead, restored real-time tracking performance, and maintained a stable thermal envelope well below the throttling limit during continuous operation.

12.2 Serial Communication Integrity and Silent Corruption

Early dynamic driving tests revealed critical instability across the serial communication buses. High-current switching from the L298N motor drivers induced severe electrical noise across the chassis wiring. While the microcontrollers remained active, packet collisions caused silent data corruption—where serial payloads appeared successfully delivered but contained altered characters. This occasionally resulted in phantom motor commands or corrupted sensor telemetry reaching the ROS 2 graph.

To counter this vulnerability, strict data validation and source coding protocols were retrofitted into the firmware pipelines:

- **XOR Checksum Validation:** An 8-bit XOR checksum loop was mandated for all ASCII telemetry frames traveling between the PIC and the ESP32. Any frame failing the checksum

parity is immediately discarded by the receiving node, preventing noise-induced phantom behavior.

- **COBS Source Coding:** On the higher-speed USB link bridging the ESP32 and the Raspberry Pi, null-byte desynchronization caused buffer parsing errors. Applying Consistent Overhead Byte Stuffing (COBS) on top of the Protocol Buffers binary stream guaranteed absolute, unambiguous frame boundaries.

The combination of these two protocols successfully rejected 100% of corrupted packets, stabilizing the internal network under aggressive locomotion.

13 Conclusion and Future Work

13.1 Project Summary

VMO demonstrates the design and execution of a resource-constrained, multi-tier autonomous mobile companion robot. By distributing processing across a strict hierarchy — hard real-time sensor polling on a PIC16F877A, asynchronous queue management on a FreeRTOS ESP32-S3, and high-level perception and navigation inside a containerized ROS 2 Raspberry Pi 4 environment — the system achieves real-time computer vision inference and reactive subsumption navigation under tight memory budgets without cloud dependency.

The NMS-free YOLO26 Nano model, combined with the Unique Frames Processing Algorithm, proves that deep learning inference can run stably on edge hardware within safe thermal bounds. Leveraging a consumer Android device as expression display, voice interface, and network gateway eliminates discrete hardware cost for OLED panels, audio amplifiers, and dedicated voice ADC boards. VMO stands as a functional prototype for an offline-first, cost-effective, distributed home assistant framework that bridges consumer social robotics with proactive safety monitoring.

13.2 Future Improvements Roadmap

The current architecture supports the following planned upgrades:

1. **Visual-Inertial Odometry (VIO):** Integrating the smartphone's MEMS accelerometer and gyroscope into a complementary filter to build a Visual-Inertial Navigation System (VINS). This will compute relative position matrices with millimeter accuracy, eliminating wheel slippage drift on smooth surfaces.

2. **Automated Vision-Tag Docking:** When the smartphone battery drops below 15%, the application will switch its camera stream to an AprilTags visual tracking filter, scan for a charging anchor sticker, compute the alignment vector, and autonomously reverse the robot onto the physical charging pins.

3. **On-Device Neural Core Acceleration (NPU):** Compiling YOLO26 Nano variants directly for the smartphone's integrated Neural Processing Unit to run parallel person-detection loops alongside the Pi's vision pipeline, reducing latency and network socket dependency for companion interaction tracking.

4. **Expanded Safety Response on the ESP32:** Currently, the PIC firmware delegates all gas alarm response to the ESP32. A planned firmware update will implement structured hazard response behaviors on the ESP32 — including audible alerts, LED strobing, and mandatory motor lock — triggered automatically when `s_gas_locked` is raised.

5. **Closed-Loop Battery Management Telemetry:** Integrating the INA226 real-time current readings into the ROS 2 graph as a dedicated topic, enabling the navigation stack to automatically reduce motor speed or return to a home position when total system draw approaches battery capacity limits.

A Appendix: System Interconnect Diagram (SID)

The master SID in Figure 9 represents the final production-grade hardware blueprint for the VMO platform. It illustrates the complete wiring architecture, power distribution tracks, parallel buck regulator divisions, logic level converter paths, and all sensor node connections.

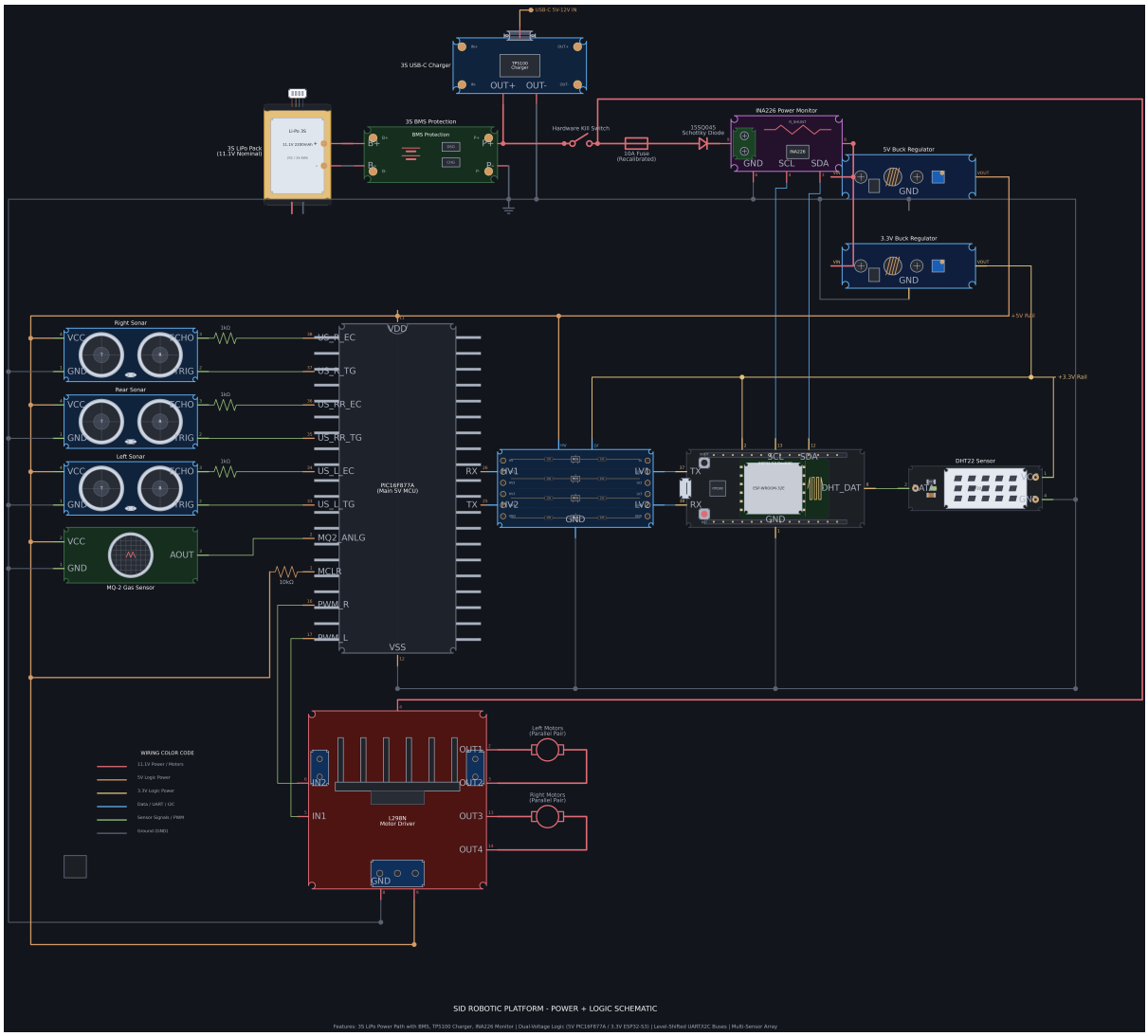

Figure 9: VMO Robotic Platform Master Power and Logic System Interconnect Diagram (SID).

The SID verifies the structural separation of concerns between low-power sensor networks and high-current motor drive axles. It highlights the logic level converter lanes bridging the PIC16F877A and ESP32-S3 processor domains, and details the decoupling networks that isolate core electronics from motor switching noise.